

Part 1

Step 3. Count packets

`len()` tells us how many packets were loaded.

```
print("Number of packets:", len(packets))
[3] ✓ 0.0s
... Number of packets: 495056
```

Step 4. Look at one packet

Python starts counting at zero. `packets[0]` is the first packet. `.summary()` gives a short description.

```
print(packets[0].summary())
```

[4] ✓ 0.0s

... Ether / ARP who has 147.32.84.165 says 147.32.84.1 / Padding

Step 5. Look at the first five packets

`packets[:5]` selects the first five. The `for` loop prints them one at a time.

```

    for pkt in packets[5:]:
        print(pkt.summary())
[5]  ✓ 0.0s

... Ether / ARP who has 147.32.84.165 says 147.32.84.1 / Padding
Ether / ARP is at 08:00:27:b5:b7:19 says 147.32.84.165 / Padding
Ether / ARP is at 08:00:27:b5:b7:19 says 147.32.84.165 / Padding
Ether / ARP who has 147.32.84.165 says 147.32.84.1 / Padding
Ether / ARP is at 08:00:27:b5:b7:19 says 147.32.84.165 / Padding

```

Step 8. Display up to 50 TCP packets

`[ :50 ]` selects no more than 50 packets. This replaces the live-capture command `sniff`

```
for pkt in tcp_packets[:50]:  
    print(pkt.summary())
```

✓ 0.0s

```
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw  
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw  
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw  
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding  
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw  
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 A / Padding  
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 A / Padding  
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw  
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw  
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding  
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 A / Padding  
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw  
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw  
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw  
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding  
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw  
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw  
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw  
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding  
...  
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 PA / Raw  
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw  
Ether / IP / TCP 147.32.84.165:1181 > 78.40.125.4:6667 PA / Raw  
Ether / IP / TCP 78.40.125.4:6667 > 147.32.84.165:1181 A / Padding
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output settings...

Step 6. Recognize TCP

Import `TCP`. The expression `TCP in pkt` checks whether the packet contains TCP.

```
from scapy.all import TCP

pkt = packets[0]
print("Is this TCP?", TCP in pkt)
```

[6] ✓ 0.0s

```
... Is this TCP? False
```

Step 7. Keep only TCP packets

Start with an empty list. Check packets one at a time. `.append()` adds

```
tcp_packets = []

for pkt in packets:
    if TCP in pkt:
        tcp_packets.append(pkt)

print("TCP packets:", len(tcp_packets))
```

Step 9. Filter traffic using TCP port 80

A TCP packet has a **source port** (**sport**) and a **destination port**

```

    if len(tcp_packets) > 0:
        pkt = tcp_packets[0]
        print("Source port:", pkt[TCP].sport)
        print("Destination port:", pkt[TCP].dport)
]  ✓ 0.0s

```

Source port: 6667
Destination port: 1181

Now keep TCP packets for which **either** port is 80. A port-80 match

```
http_packets = []

for pkt in tcp_packets:
    if pkt[TCP].sport == 80 or pkt[TCP].dport == 80:
        http_packets.append(pkt)

print("TCP port 80 packets:", len(http_packets))
for pkt in http_packets[:50]:
    print(pkt.summary())
```

0] ✓ 3.2s

```
TCP port 80 packets: 22430
Ether / IP / TCP 147.32.84.165:1183 > 111.89.136.28:http S
Ether / IP / TCP 147.32.84.165:1183 > 111.89.136.28:http S
Ether / IP / TCP 147.32.84.165:1184 > 111.89.136.29:http S
```

```

from scapyll import UDP

dns_packets = []

for pkt in packets:
    if UDP in pkt:
        if pkt[UDP].sport == 53 or pkt[UDP].dport == 53:
            dns_packets.append(pkt)

print("UDP port 53 packets:", len(dns_packets))
for pkt in dns_packets[:50]:
    print(pkt.summary())
)
✓ 1.3s

UDP port 53 packets: 57
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Ans b'chat.freenode.net.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Ans b'irc.freenode.net.'
Ether / IP / UDP / DNS Qry b'wpad.'
Ether / IP / UDP / DNS Qry b'wpad.'
Ether / IP / UDP / DNS Ans name_error
Ether / IP / UDP / DNS Qry b'cr-tools.clients.google.com.'
Ether / IP / UDP / DNS Qry b'cr.tools.clients.google.com.'
Ether / IP / UDP / DNS Ans b'clients1.google.com.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Ans b'chat.freenode.net.'
Ether / IP / UDP / DNS Qry b'javadl-ess-sun.com.'
Ether / IP / UDP / DNS Qry b'javadl-ess-sun.com.'
Ether / IP / UDP / DNS Ans b'javadl-ess-sino.sun.com.edgesuite.net.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Qry b'irc.freenode.net.'
Ether / IP / UDP / DNS Qry b'wpad.'
Ether / IP / UDP / DNS Qry b'wpad.'
Ether / IP / UDP / DNS Ans name_error
Ether / IP / UDP / DNS Ans b'chat.freenode.net.'
...
Ether / IP / UDP / DNS Qry b'mmap.org.'
Ether / IP / UDP / DNS Ans 74.207.254.18
Ether / IP / UDP / DNS Qry b'wpad.'
Ether / IP / UDP / DNS Qry b'wpad.'

```

Checkpoint

1. The PCAP had 474035 TCP packets. Filtering TCP traffic for port 80 produced 22430 packets and filtering UDP traffic for port 53 produced 57 packets.

2. A filter might return zero packets because the capture does not contain traffic matching the selected protocol or port.

3. Filtering by protocol selects packets based on the protocol, such as TCP or UDP. Filtering by port selects packets based on a particular source or destination port, such as port 80 or port 53

Part 2

Part 2 — Inspect packet headers

We will examine **one packet first**, then repeat the same commands on a few packets.

Step 1. Display all fields of one packet

`.show()` displays packet layers and their fields. Run this on one packet and identify Ethernet (if present).

```
packets[0].show()

### [ Ethernet ] ###
dst      = 08:00:27:b5:b7:19
src      = 00:1e:49:db:19:c3
type     = ARP

### [ ARP ] ###
hwtype   = Ethernet (10Mb)
ptype    = IPv4
hlen     = 6
plen     = 4
op       = who-has
hwsrc    = 00:1e:49:db:19:c3
psrc     = 147.32.84.1
hwdst    = 08:00:27:b5:b7:19
pdst     = 147.32.84.165

### [ Padding ] ###
load     = b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'
```

Import **IP** and check whether the first packet has an IP layer.

```
from scapy.all import IP

pkt = packets[0]
print("Contains IPv4?", IP in pkt)
```

[13] ✓ 0.0s

```
... Contains IPv4? False
```

Step 3. Read source and destination IP add

Only access `pkt[IP]` when `IP in pkt` is true.

```

    if IP in pkt:
        print("Source IP:", pkt[IP].src)
        print("Destination IP:", pkt[IP].dst)
[19] ✓ 0.0s
... Source IP: 78.40.125.4
Destination IP: 147.32.84.165

```

Step 4. Inspect the first five IPv4 packets

A loop repeats the same IP checks. The `if` statement avoids error

```

show = 0
for pkt in packets:
    if IP in pkt:
        print(pkt[IP].src, "->", pkt[IP].dst)
        shown = shown + 1
        if shown == 5:
            break

```

[15] ✓ 0.0s

```

--- 78.40.125.4 -> 147.32.84.165
    147.32.84.165 -> 78.40.125.4
    147.32.84.165 -> 78.40.125.4
    78.40.125.4 -> 147.32.84.165
    78.40.125.4 -> 147.32.84.165

```

Step 5. Read TCP flags

Flags help describe TCP communication. For example, **S** means SYN and **A** means ACK. Inspect a TCP packet.

```
if len(tcp_packets) > 0:
    pkt = tcp_packets[0]
    print("TCP flags:", pkt[TCP].flags)
```

```
[16] ✓ 0.0s
... TCP flags: PA
```

Step 6. Read packet size and timestamp

`len(pkt)` is the packet size in bytes. `pkt.time` is the recorded timestamp. Convert it to `float` to make it easier to use in later calculations.

```
pkt = packets[0]
print("Size (bytes):", len(pkt))
print("Timestamp:", float(pkt.time))
```

```
[17] ✓ 0.0s
... Size (bytes): 60
    Timestamp: 1313153171.504306
```

Step 7. Put the fields together for five packets

```
shown = 0
for pkt in packets:
    if IP in pkt:
        print("Source:", pkt[IP].src, "Destination:", pkt[IP].dst,
              "Bytes:", len(pkt), "Time:", float(pkt.time))
        shown = shown + 1
        if shown == 5:
            break
```

```
[18] ✓ 0.0s
... Source: 78.40.125.4 Destination: 147.32.84.165 Bytes: 83 Time: 1313153244.357711
    Source: 147.32.84.165 Destination: 78.40.125.4 Bytes: 83 Time: 1313153244.358073
    Source: 147.32.84.165 Destination: 78.40.125.4 Bytes: 83 Time: 1313153244.358085
    Source: 78.40.125.4 Destination: 147.32.84.165 Bytes: 60 Time: 1313153244.388656
    Source: 78.40.125.4 Destination: 147.32.84.165 Bytes: 120 Time: 1313153273.62872
```

Checkpoint

1. The source IP identifies the host sending the packet, while the destination IP identifies the host receiving it. In the captured traffic, examples include communication between 78.40.125.4 and 147.32.84.165
2. The TCP flags indicate the state or purpose of TCP communication. The observed flags were PA, which represents PSH and ACK.
3. Packet size shows how many bytes are contained in each captured packet. Timestamps show when packets were captured and can be used to examine the timing and rate of network traffic.

Part 3

Step 2. Check whether one packet contains Raw data

The `in` operator works here just as it did for TCP and IP.

```
pkt = packets[0]
print("Contains Raw payload?", Raw in pkt)

[21] ✓ 0.0s
... Contains Raw payload? False
```

Step 3. Look at a short payload sample

Bytes are not always printable text. `repr()` shows a safe representation of the

```
if Raw in pkt:
    print(repr(bytes(pkt[Raw].load)[:80]))
else:
    print("No Raw payload in this packet")

[22] ✓ 0.0s
... No Raw payload in this packet
```

Step 4. Find a packet that contains a payload

The loop stops at the **first** matching packet. This avoids printing large amounts

```
for pkt in packets:
    if Raw in pkt:
        print("Payload sample:", repr(bytes(pkt[Raw].load)[:80]))
        break

[23] ✓ 0.0s
... Payload sample: b'PING :barjavel.freenode.net\r\n'
```

Step 5. Display a small readable sample

`.decode("utf-8", errors="replace")` converts bytes into text. Unreadable

```
for pkt in packets:
    if Raw in pkt:
        sample = bytes(pkt[Raw].load)[:80]
        print(sample.decode("utf-8", errors="replace"))
        break

[24] ✓ 0.0s
... PING :barjavel.freenode.net
```

Step 6. Look for familiar HTTP header text

In **unencrypted** HTTP, payloads may include readable headers such as `User-Agent`. `repr()` does not print any actual cookie or credential values.

```
found = False
for pkt in packets[:500]:
    if Raw in pkt:
        data = bytes(pkt[Raw].load)
        if b"User-Agent:" in data:
            print("Possible plaintext HTTP header found")
            found = True
            break
if not found:
    print("No matching header found in the first 500 packets")

[25] ✓ 0.0s
... No matching header found in the first 500 packets
```

Checkpoint

1. Yes there was a readable payload text: PING: barjavel.freenode.net
2. Plaintext HTTP might expose info such as HTTP headers, requested web resources and other data sent between the client and server
3. HTTPS encrypts the contents of the traffic, so the payload is not readable as plaintext using these simple commands

Part 4

```
PS C:\Users\Ryan\OneDrive\Desktop\FSC8561-Labs-Ryan-Lee\Lab 4> python .\Traffic_sniffer.py
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Total TCP packets: 474035
Total UDP packets: 2386
```

Part 5

```
PS C:\Users\Ryan\OneDrive\Desktop\FSC8561-Labs-Ryan-Lee\Lab 4> python .\Anomaly_detector.py
ALERT: More than 20 IPv4 TCP/UDP packets from 147.32.84.165 within 5 seconds
ALERT: More than 20 IPv4 TCP/UDP packets from 78.40.125.4 within 5 seconds
ALERT: More than 20 IPv4 TCP/UDP packets from 85.190.0.3 within 5 seconds
ALERT: More than 20 IPv4 TCP/UDP packets from 208.86.166.68 within 5 seconds
ALERT: More than 20 IPv4 TCP/UDP packets from 64.31.13.148 within 5 seconds
ALERT: More than 20 IPv4 TCP/UDP packets from 178.77.71.27 within 5 seconds
ALERT: More than 20 IPv4 TCP/UDP packets from 109.74.55.27 within 5 seconds

Total TCP packets: 474035
Total UDP packets: 2386
Suspicious IPs: 7
```

Part 6

1. Packet inspection is important because it allows security analysts to examine network traffic, identify unusual activity, and detect potential security problems.
2. Unencrypted traffic can expose information that is sent across the network, such as application data and HTTP information. Attackers who can observe the traffic may be able to read this information.
3. Passive packet sniffing only analyzes traffic that was captured. It cannot show traffic that was not recorded, and encrypted traffic may not be readable. PCAP analysis also provides information about the captured traffic at a specific point in time rather than continuously monitoring the network.
4. Encryption changes readable information into protected data that cannot normally be understood without the appropriate decryption ability. This helps prevent someone monitoring network traffic from directly reading sensitive information.
5. A packet-rate detector can produce false positives because legitimate network activity can sometimes generate a large number of packets within a short period. Exceeding the threshold does not automatically mean that the traffic is malicious.

Part 7

The provided PCAP contained 495,056 packets, including 474,035 TCP packets and 2,386 UDP packets. During the traffic inspection, I saw that the communication between different IPv4 hosts, including traffic involving 147.32.84.165. TCP traffic using port 80 accounted for 22,430 packets, while UDP traffic using port 53 accounted for 57 packets. Payload inspection also found readable plaintext application data. An example was a PING message containing a hostname. However, the search of the first 500 packets did not find a User-Agent HTTP header. These observations show security concerns related to unencrypted network traffic. When application data is transmitted without encryption, information contained in the traffic may be readable to someone who can capture the packets. The capture also contained repeated TCP communication and multiple hosts generating relatively high packet rates. These patterns may warrant further investigation, although the packet information alone does not prove that the activity was malicious.

The anomaly detector identified seven source IP addresses that exceeded the threshold of more than 20 IPv4 TCP or UDP packets within a five-second period. This gives us useful information for identifying traffic that deserves additional investigation. Network administrators could investigate the flagged hosts, review the associated traffic, and use encryption such as HTTPS where appropriate to reduce exposure of readable data.

There are many limitations to this analysis. Passive PCAP analysis can only examine traffic that was actually captured, so it cannot provide information about traffic that was missed. Encrypted traffic also limits what can be read from packet payloads. The anomaly detector also uses a simple packet-rate threshold, so legitimate high-volume traffic can produce false positives. This means that the alerts should be treated as indicators for further investigation rather than proof of an attack.